Эффективная реализация лямбда-функций в объектно-ориентированных языках программирования

Болохононов Артем
Научный руководитель: Трепаков Иван Сергеевич

Анонимные функции

Определение

```
def filterEven(x: Int): Bool = {
  x % 2 == 0
}
```

Анонимные функции

Определение

- *Анонимные функции* (или *лямбда-функции*) — функции, которые создаются по ходу программы и не имеют уникального индентификатора,

```
val filerEvenLambda: Int ⇒ Bool = { x: Int ⇒ x % 2 }
```

Анонимные функции

Определение

- *Анонимные функции* (или *лямбда-функции*) — функции, которые создаются по ходу программы и не имеют уникального индентификатора,
- Анонимные функции могут замыкать окружающий контекст, в таком случае они называются *функциональными замыканиями*,

```
val filerEvenLambda: Int ⇒ Bool = { x: Int ⇒ x % 2 }
```

```
val capture: Int = 5
val functionalCaputre: ⇒ Int = { capture }

...

functionalCapture() // == 5
```

Анонимные функции

Определение

- *Анонимные функции* (или *лямбда-функции*) — функции, которые создаются по ходу программы и не имеют уникального индентификатора,

```
val filerEvenLambda: Int ⇒ Bool = { x: Int ⇒ x % 2 }
```

- Анонимные функции могут замыкать окружающий контекст, в таком случае они называются *функциональными замыканиями*,

```
val capture: Int = 5
val functionalCaputre: ⇒ Int = { capture }

...

functionalCapture() // == 5
```

- Сейчас в объектно-ориентированных языках программирования они используются повсеместно, часто используются при работе с коллекциями.

```
val collection: Seq[Int] = ...
collection.filter(x ⇒ x % 2)
collection.sort(x, y ⇒ x > y)
collection.foreach(x ⇒ println(x))
collection.map(x ⇒ x * x)
...
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
val lambda = { x ⇒ println(x) }
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
class Lambda$$1 extends (Int ⇒ Unit) {
  override def apply(x: Int) = {
    println(x)
  }
}

val lambda = { x ⇒ println(x) }
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
class Lambda$$1 extends (Int ⇒ Unit) {
  override def apply(x: Int) = {
    println(x)
  }
}

val lambda = new Lambda$$1()
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
val capture = "arg = "

</d
val lambda = { x => println(capture + x) }
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
class Lambda$$1(val prefix: Str) extends (Int ⇒ Unit) {
  override def apply(x: Int) = {
    println(prefix + x)
  }
}

val capture = "arg = "
val lambda = new Lambda$$1(capture)
lambda(42)
```

Постановка задачи

Проблема

Каждая из анонимных функций в условиях *JVM* (*Java Virtual Machine*) — это полноценный объект, который требует как аллокации, так и сборки мусора. Из-за чего возникают издержки по памяти и времени.

Постановка задачи

Проблема

Каждая из анонимных функций в условиях *JVM* (*Java Virtual Machine*) — это полноценный объект, который требует как аллокации, так и сборки мусора. Из-за чего возникают издержки по памяти и времени.

Безусловно, существует набор оптимизаций, которые могут помочь в избавлении от создания объектов в куче.

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

1. Скаляризация объектов

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = SomeObject(42, 42.0, "42")
useField(obj.a)
useField(obj.b)
obj.c = "override"
```

Оптимизации

Анализ утеканий

1. Скаляризация объектов

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"
```

Оптимизации

Анализ утеканий

1. Скаляризация объектов

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"
```

- Данное преобразование может быть применено только на объектах, у которых нет использований по ссылке

Оптимизации

Анализ утеканий

2. Создание объектов на стеке

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = SomeObject(42, 42.0, "42")
useField(obj.a)
useField(obj.b)
obj.c = "override"

useObject(obj)
```

Оптимизации

Анализ утеканий

2. Создание объектов на стеке

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = onStack(SomeObject(42, 42.0, "42"))
useField(obj.a)
useField(obj.b)
obj.c = "override"

useObject(obj)
```

Оптимизации

Анализ утеканий

2. Создание объектов на стеке

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = onStack(SomeObject(42, 42.0, "42"))
useField(obj.a)
useField(obj.b)
obj.c = "override"

useObject(obj)
```

- Можно применить, если доказанно, что объект существует только в пределах своего стекового кадра

Оптимизации

Анализ эвакуаций

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = onStack(SomeObject(42, 42.0, "42"))
useField(obj.a)
useField(obj.b)
obj.c = "override"

otherObject.field = obj  // otherObject также находится на стеке
useObject(obj)

...

staticField = heapification(otherObject)
```

Оптимизации

Анализ эвакуаций

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = onStack(SomeObject(42, 42.0, "42"))
useField(obj.a)
useField(obj.b)
obj.c = "override"

otherObject.field = obj  // otherObject также находится на стеке
useObject(obj)

...

staticField = heapification(otherObject)
```

- Мощный оптимистичный статический анализ, который обходит весь граф вызовов

Оптимизации

Анализ эвакуаций

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = onStack(SomeObject(42, 42.0, "42"))
useField(obj.a)
useField(obj.b)
obj.c = "override"

otherObject.field = obj  // otherObject также находится на стеке
useObject(obj)

...

staticField = heapification(otherObject)
```

- Мощный оптимистичный статический анализ, который обходит весь граф вызовов
- Однако обладает большими издержками: увеличение времени компиляции до 300 раз

Оптимизации

Стратегии

1. Анализ утеканий
2. Анализ эвакуаций

Оптимизации

Стратегии

1. Анализ утеканий
2. Анализ эвакуаций
3. Частичный анализ эвакуаций

Тестовый стенд

Huawei VM

Проблематика

Цель и задачи

Цель: разработать и реализовать оптимизацию лямбда-функций на основе анализа утеканий, которая уменьшит потребление памяти за счет увеличения количества размещений анонимных функций на стек,

Проблематика

Цель и задачи

Цель: разработать и реализовать оптимизацию лямбда-функций на основе анализа утеканий, которая уменьшит потребление памяти за счет увеличения количества размещений анонимных функций на стек,

Задачи:

1. Изучить существующие подходы
2. Изучить текущую реализацию в компиляторе **Huawei VM**
3. Разработать и реализовать оптимизацию
4. Протестировать и замерить результаты

Частичный анализ эвакуаций

Идея

```
def lambdaUse(x: (Int ⇒ Unit)) = {
  ...
  if (globalFlag) {  // Всегда == false
    globalVar = x
  }
  ...
}

val lambda = { x ⇒ println(x) }
lambdaUse(lambda)
```

Частичный анализ эвакуаций

Идея

```
def lambdaUse(x: (Int ⇒ Unit)) = {
  ...
  if (globalFlag) {  // Всегда == false
    globalVar = evacuate(x)
  }
  ...
}

val lambda = onStack( { x ⇒ println(x)} )
lambdaUse(lambda)
```

Частичный анализ эвакуаций

Идея

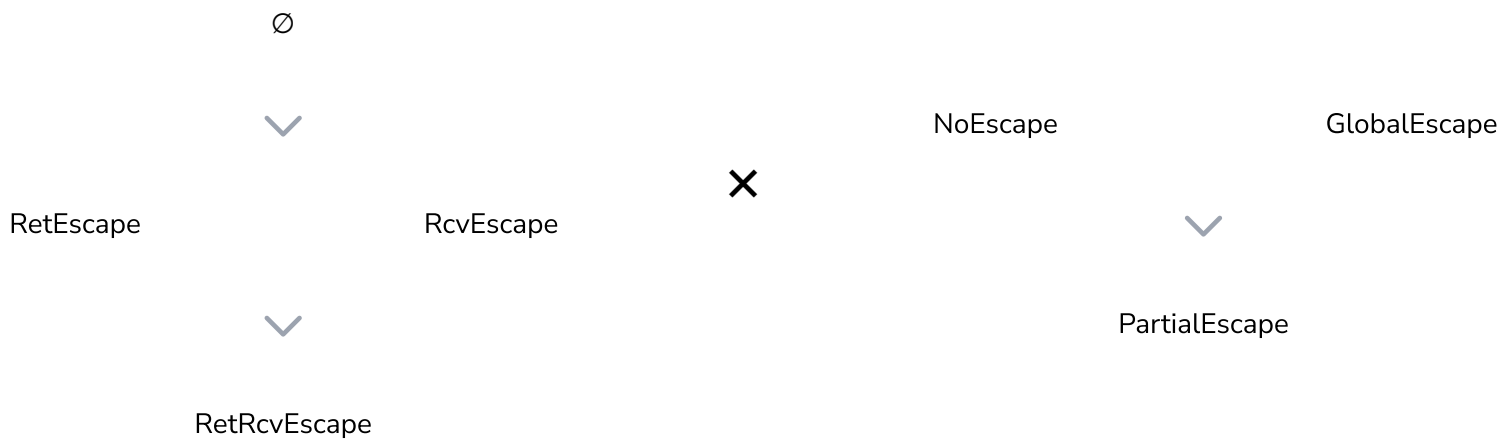
```
def lambdaUse(x: (Int ⇒ Unit)) = {
  ...
  if (globalFlag) {  // Всегда == false
    globalVar = evacuate(x)
  }
  ...
}

val lambda = onStack( { x ⇒ println(x)} )
lambdaUse(lambda)
```

- Будем перемещать объект на стек, если у него нет гарантированного *утекающего* использования
- Если такие использования есть, то перед их исполнением будем вставлять операцию *эвакуация*, которая создаст копию объекта на куче

Анализ утеканий

Алгоритм

Частичный анализ эвакуаций

Алгоритм